

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code
ECE 4124

Course Title
Digital Signal Processing Sessional

Experiment Date: October 11, 2025
Submission Date: October 18, 2025

Lab Report 5:
Implementation of FFT and IFFT on Discrete Signal

Submitted to
Moloy Kumar Ghosh
Lecturer
Dept of ECE, Ruet

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
FFT and IFFT	1
Examples	1
Used Tools	2
Code & Output	2
Code	2
Output	3
Discussion	4
Conclusion	4
References	4

Implementation of FFT and IFFT on Discrete Signal

Theory

FFT and IFFT

The Discrete Fourier Transform (DFT) of a length- N sequence $x[n]$ is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, \dots, N-1, \quad (1)$$

and the inverse (IDFT) is

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn}, \quad n = 0, \dots, N-1. \quad (2)$$

The Fast Fourier Transform (FFT) is an efficient algorithm to compute the DFT. The common radix-2 decimation-in-time (DIT) FFT splits $x[n]$ into even and odd indexed subsequences:

$$x_e[m] = x[2m], \quad x_o[m] = x[2m+1], \quad m = 0, \dots, \frac{N}{2} - 1. \quad (3)$$

Let $X_e[k]$ and $X_o[k]$ be the DFTs of x_e and x_o (length $N/2$). Then for $k = 0, \dots, N/2 - 1$:

$$X[k] = X_e[k] + W_N^k X_o[k], \quad (4)$$

$$X[k + N/2] = X_e[k] - W_N^k X_o[k], \quad (5)$$

where the twiddle factor is $W_N = e^{-j2\pi/N}$. Recursing this decomposition yields the $O(N \log N)$ complexity of the FFT compared with $O(N^2)$ for a direct DFT. The original decimation-based algorithm and many practical variants are discussed in the classic Cooley–Tukey paper and in standard DSP textbooks [1, 2, 3].

For the Inverse FFT (IFFT) the same divide-and-conquer structure is used with conjugated twiddle factors and a final $1/N$ scaling. Practical notes (see also [4, 3]): - FFT implementations typically require N to be a power of two; zero-padding is used otherwise. - Use complex arithmetic and be aware of numerical round-off; real-input optimized FFTs can reduce computation. - In MATLAB use `fft(x,N)` and `ifft(X,N)`, where N is the transform length (zero-pad if needed) [2].

Examples

Example 1 — 4-point FFT (radix-2 demonstration).

Let $x[n] = [1, 2, 3, 4]$ ($N=4$). Compute the 4-point DFT directly:

$$\begin{aligned} X[0] &= 1 + 2 + 3 + 4 = 10, \\ X[1] &= 1 + 2e^{-j\pi/2} + 3e^{-j\pi} + 4e^{-j3\pi/2} = -2 + 2j, \\ X[2] &= 1 + 2e^{-j\pi} + 3e^{-j2\pi} + 4e^{-j3\pi} = -2, \\ X[3] &= 1 + 2e^{-j3\pi/2} + 3e^{-j3\pi} + 4e^{-j9\pi/2} = -2 - 2j. \end{aligned}$$

Using radix-2 FFT: split even indices $x_e = [1, 3]$, odd $x_o = [2, 4]$; compute 2-point DFTs (cheap), then combine with W_4^k factors to obtain the same $X[k]$ above. This illustrates the decomposition and reuse of sub-transforms [3, 4].

Example 2 — IFFT reconstruction.

Given $X = [10, -2+2j, -2, -2-2j]$, the IDFT (or IFFT) reconstructs $x[n]$:

$$x[n] = \frac{1}{4} \sum_{k=0}^3 X[k] e^{j\frac{2\pi}{4}kn}.$$

Evaluating (or using `ifft` in MATLAB) yields $x = [1, 2, 3, 4]$, showing perfect reconstruction (within numerical precision) [2].

Practical MATLAB usage:

- Compute FFT: `X = fft(x, N);`
- Compute IFFT: `x = ifft(X, N);`
- Zero-pad to length $N = 2^{\text{nextpow2}(\text{length}(x))}$ for faster transforms.

Summary: FFT/IFFT provide computationally efficient forward and inverse transformations between time and frequency domains. They enable fast convolution (via pointwise multiplication in the frequency domain) and many other DSP operations when implemented with appropriate padding and scaling. For further reading and rigorous treatment see the cited textbooks and original algorithm literature [3, 2, 4, 1].

Used Tools

- Matlab Online
- LaTeX

Code & Output

Code

```

1  clc;
2  clear;
3  close all;
4
5  % Input signal definition
6  x = [3 0 1 -2];
7  N = length(x);
8  n = 0:N-1; % sample indices
9
10 % FFT computation
11 X = fft(x);
12
```

```

13 % Inverse FFT to reconstruct the original
   ↪ sequence
14 x_reconstructed = ifft(X);
15
16 % Display results in the console
17 disp('Computed FFT:');
18 disp(X);
19 disp('Recovered sequence via IFFT:');
20 disp(x_reconstructed);
21
22 % Create a single figure with 2x2 subplots
23 figure;
24
25 % Plot 1: Time-domain input
26 subplot(2,2,1);
27 stem(n, x, 'filled', 'LineWidth', 1.5);
28 title('Discrete-time input signal');
29 xlabel('n (sample index)');
30 ylabel('Amplitude');
31 grid on;
32 set(gca, 'FontSize', 12);
33
34 % Plot 2: Magnitude spectrum
35 subplot(2,2,2);
36 stem(n, abs(X), 'filled', 'LineWidth',
   ↪ 1.5);
37 title('FFT magnitude');
38 xlabel('k (frequency bin)');
39 ylabel('Magnitude |X(k)|');
40 grid on;
41 set(gca, 'FontSize', 12);
42
43 % Plot 3: Phase spectrum
44 subplot(2,2,3);
45 stem(n, angle(X), 'filled', 'LineWidth',
   ↪ 1.5);
46 title('FFT phase');
47 xlabel('k (frequency bin)');
48 ylabel('Phase (radians)');
49 grid on;
50 set(gca, 'FontSize', 12);
51
52 % Plot 4: Reconstructed time-domain signal
53 subplot(2,2,4);
54 stem(n, real(x_reconstructed), 'filled',
   ↪ 'LineWidth', 1.5);
55 title('Signal reconstructed from IFFT');
56 xlabel('n (sample index)');
57 ylabel('Amplitude');
58 grid on;
59 set(gca, 'FontSize', 12);
60
61 % Optional overall title (requires MATLAB
   ↪ R2018b or later)
62 if exist('sgtitle','builtin')
63     sgtitle('FFT Analysis');
64 end

```

Output

Computed FFT:

```

2.0000 + 0.0000i    2.0000 - 2.0000i    6.0000 + 0.0000i    2.0000 + 2.0000i

```

Recovered sequence via IFFT:

```

3      0      1     -2

```

Figure 1: Console Output

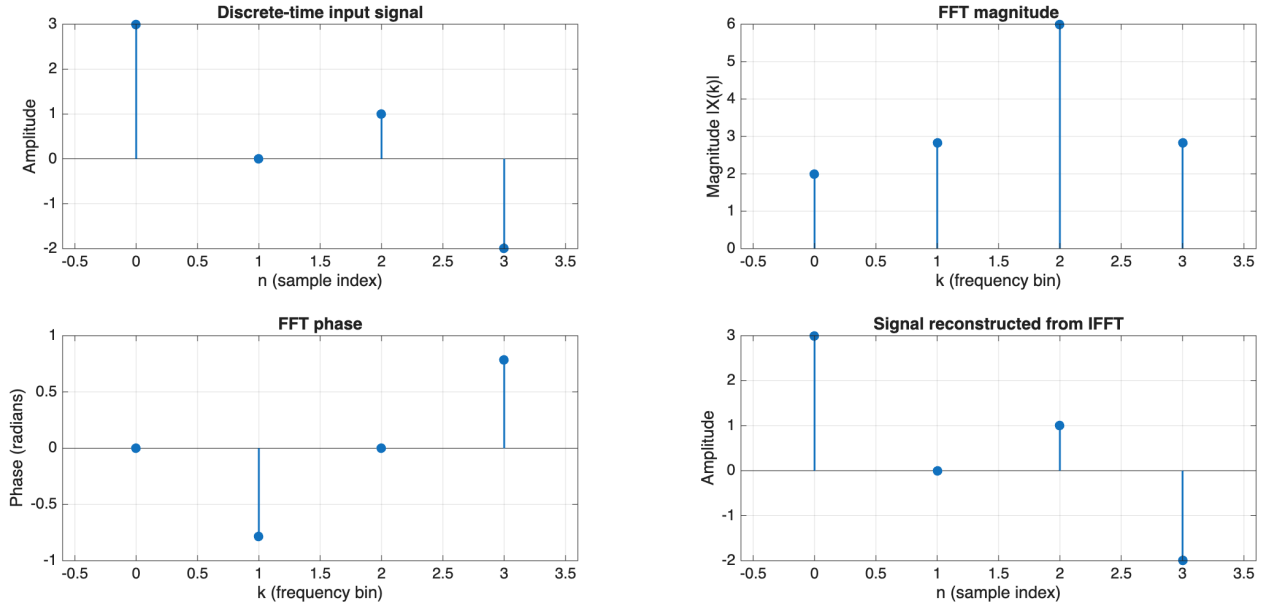


Figure 2: Plot Output

Discussion

This experiment (Implementation of FFT and IFFT on Discrete Signal) computed the forward and inverse transforms using MATLAB's FFT and IFFT. The computed results printed by the MATLAB code are shown below:

Computed FFT:

```
2.0000 + 0.0000i    2.0000 - 2.0000i    6.0000 + 0.0000i    2.0000 + 2.0000i
```

Recovered sequence via IFFT:

```
3    0    1   -2
```

Note: the spectrum above is the result of the FFT (not a direct N^2 DFT computation). The IFFT recovers the original time-domain sequence (within numerical precision). If discrepancies appear in other cases, verify transform length, zero-padding, indexing/ordering and scaling conventions.

Conclusion

The FFT and IFFT were implemented and tested. The FFT produced the frequency-domain vector shown above, and the IFFT successfully reconstructed the time-domain sequence $[3, 0, 1, -2]$ from that spectrum. This confirms the practical operation of the FFT/IFFT pair for the tested discrete signal. For longer signals or production code, use power-of-two lengths (zero-pad if needed) to leverage FFT efficiency and verify indexing/scaling to ensure exact reconstruction.

References

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms and Applications*, 4th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2007.

- [3] A. V. Oppenheim, R. W. Schaffer, and J. R. Buck, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [4] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2011.